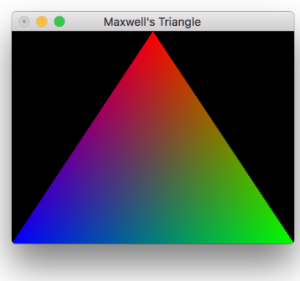
Homework 2: 2D Graphics

CS 440 Computer Graphics
Habib University
Spring 2026

Due: 11:59pm on Friday, 13 February

In assignment 1, you created constructor functions for vectors, matrices, and their operations. Compile them all in a single file called `common.js` (including the `map_point` function). Include that file as part of all questions for this assignment, as you would need those functions for the problems in this assignment. In Problems 4 and 5, you may also reuse functions from your previous assignment.

1. Maxwell's Triangle



James Clerk Maxwell is best known for his work in electromagnetic radiation resulting in the well known Maxwell's equations.
Not surprisingly, he was also among the first ones to explore the trichromatic theory of color. This has led to what is now known as Maxwell's triangle, simply called the color triangle. It is a triangle with the primary colors at its vertices and its interior shaded by linear combinations of the colors at the vertices. At any interior point, the amount of color contributed by a vertex is proportional to the distance of the vertex from that point.

Task: WebGL interpolates the colors of the vertices across the primitive by default. Use this property to display Maxwell's triangle: a triangle whose vertices are red, green, and blue.

Files: `maxwell.html, maxwell.js`

2. Triangle in a Triangle in a Triangle ...

The Sierpinski triangle is a self-similar geometric shape with many interesting properties and can be generated in several different ways. Sample iterations of the Sierpinski triangle are shown in the Figure below. One way to construct the Sierpinski triangle is through recursion, starting with an equilateral triangle.

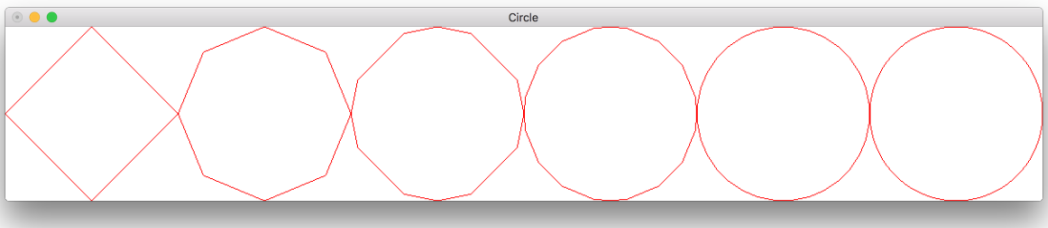
1. Start with an equilateral triangle and subdivide each of its edges and connect all its edge midpoints. This yields 4 new triangles.
2. Remove the central triangle (see triangle 2 from the left).
3. Repeat step 2 for each of the remaining triangles. At every iteration, the central triangle of each subdivision is removed.

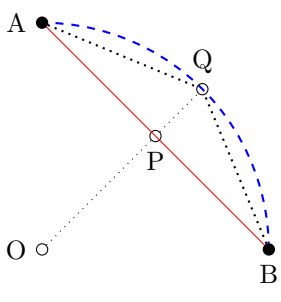
Task: Write a program to generate the Sierpinski triangle at any desired recursion level.

Files: `sierpinski.html`, `sierpinski.js`

Figure 1: Sierpinski triangle at recursion level 1, 2, 3, 4 and 5 (from left to right) respectively

3. The Smoothness Illusion: $\lim_{n\to\infty} \diamond = \circ$



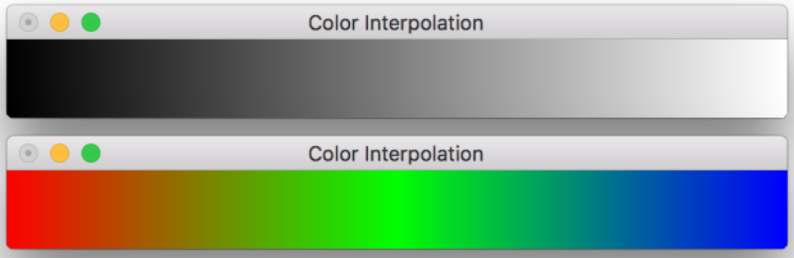


Smooth surface on a discrete screen are drawn through approximation. That is, we display an approximation of the smooth surface which, with repeated subdivision, tends to the smooth surface. For example, a circle is approximated above by successively subdivided polygons. Each subdivision step inserts an edge midpoint at each edge and projects it to the circumference of the circle being approximated. This is illustrated in the figure on the left. The red edge AB approximates the blue arc AB of the circle centered at O. In the subdivision step, the midpoint P of the edge AB is projected to Q on the arc. The edge AB is then replaced by AQB.

Task: Write a program to approximate a circle, as shown in Figure above. Your program should allow the user to specify the radius of the circle and the number of subdivision steps. You may choose any reasonable dimensions and coloring. A subdivision level of zero should display only the initial diamond. All subdivision levels (up to the user specified level) should be displayed side by side.

Files: `circle.html`, `circle.js`

4. Color Interpolation



Our Maxwell's Triangle in Problem 1 relied on the GPU's native color interpolation. For this problem,

you will draw color bars for which you will perform the interpolation using the `map_point()` function from the previous homework.

Task: Display the colorbars above in your display window with the grayscale bar above the color bar. Each bar is comprised of vertical lines rendered next to each other, i.e. on adjacent pixels, in the display window. Both bars have the same dimensions. The width has to be sufficiently large to include every color in the following RGB ranges.

- (0,0,0) to (255, 255, 255), and
- (255,0,0) to (0, 255, 0) to (0, 0, 255).

Files: `colorbar.html`, `colorbar.js`

5. The Mandelbrot Set

The sequence z_n for a complex number, c, is defined for $n \geq 0$ as

$$z_n(c) = \begin{cases} 0 & n = 0 \\ z_{n-1}^2(c) + c & \text{otherwise} \end{cases}$$

The Mandelbrot set, M, is the set of all complex numbers, c, for which $z_n(c)$ is *bounded*, i.e. $|z_n|$ is finite for all n. It can be shown that if for some $n = n_0$, $|z_{n_0} > 2|$, then the sequence z_n *escapes to infinity*, i.e. $|z_n|$ is not bounded for $n > n_0$. This leads to the conclusion that $\forall c \in M : z_n(c) \leq 2$. Since $z_1 = c$, the above condition also implies that $|c| \leq 2$. The above yields the following two handy conditions whose fulfillment by a complex number, c, establishes its membership in M:

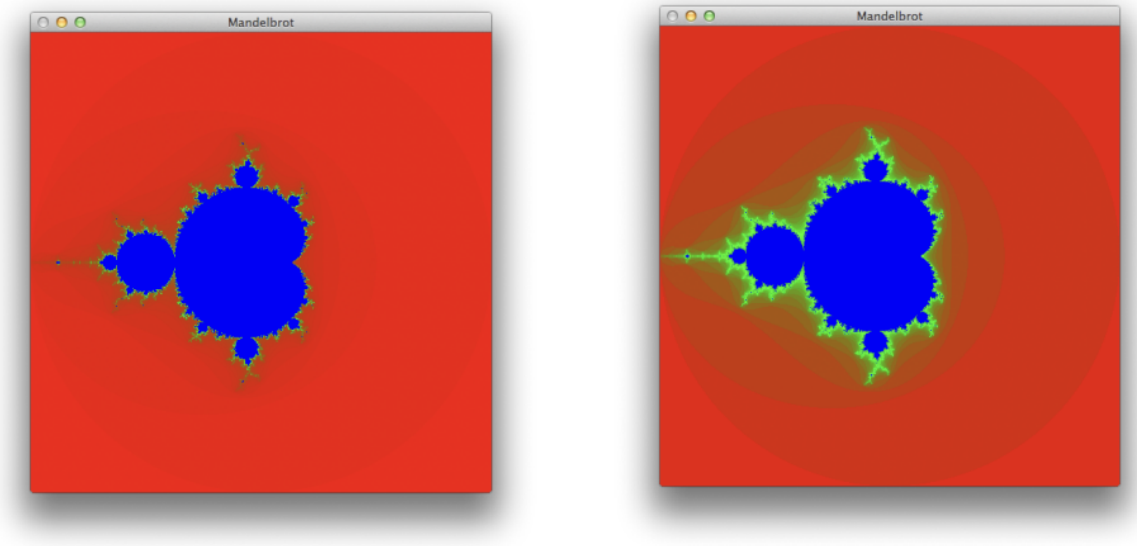
1. $|c| \leq 2$
2. $\forall n \geq 0 : |z_n(c)| \leq 2$

The value of n for which condition 2 becomes false for a given c is dubbed the *escape time* of c. Condition 2 also implies that all members of the Mandelbrot set lie within the circle in complex plane centered at the origin and with radius 2.

Example: Consider the following complex numbers and the corresponding sequence for each up to z_5.

	$c = (3+4j)$	$c = (1+1j)$	$c = (0.5+0.5j)$	$c = (-0.5-0.5j)$
$z_0(c)$	0	0	0	0
$\lvert z_0(c)\rvert$	0	0	0	0
$z_1(c)$	$(3+4j)$	$(1+1j)$	$(0.5+0.5j)$	$(-0.5-0.5j)$
$\lvert z_1(c)\rvert$	5.0	1.41	0.71	0.71
$z_2(c)$		$(1+3j)$	$(0.5+1j)$	$(-0.5+0j)$
$\lvert z_2(c)\rvert$		3.16	1.12	0.5
$z_3(c)$			$(-0.25+1.5j)$	$(-0.25-0.5j)$
$\lvert z_3(c)\rvert$			1.52	0.56
$z_4(c)$			$(-1.69-0.25j)$	$(-0.69-0.25j)$
$\lvert z_4(c)\rvert$			1.71	0.73
$z_5(c)$			$(3.29+1.34j)$	$(-0.09-0.16j)$
$\lvert z_5(c)\rvert$			3.55	0.18

- We see that $z_1 = c$. This follows from the definition of z_n.
- For the first number, $(3+4j)$, we see that $|z_1| > 2$. We can therefore say that z_n will escape to infinity and the escape time of $(3+4j)$ is 1. This expected as $(3+4j)$ lies outside the circle at the origin with radius 2.
- The second number, $(1+1j)$, lies within the required circle. But it escapes at n=2.
- The escape time for the third number, which is also inside the circle, is 5.



(a) (b)

Figure 2: The Mandelbrot set up to $n_t = 100$. The points at the corners of the windows are outside the circle of radius 2, and z_n escapes for these points at $n = 1$. They are completely red. The points in blue toward the center of the image are the ones for which z_n has not yet escaped. (a) Uniform color interpolation from Red to Green to Blue. (b) Adjusted color interpolation on the same data.

- The fourth number in also inside the circle and has not escaped until n=5. It may escape later or it may not. We cannot say.

Problem: We want to visualize the Mandelbrot set up to a certain value of $n = n_t$ in an image. We will map to our viewport the circle in the complex plane which is centered at the origin and has a radius of 2. This will yield the complex number corresponding to each pixel in the viewport. We will color each pixel according to the escape time of its corresponding complex number. In Figure 2, complex numbers that escape early are assigned red and those that have not escaped till $n = n_t$ are assigned blue. Complex numbers with intermediate escape values are assigned interpolated colors.

Thus, following steps will be involved in your Mandelbrot algorithm:

1. Map the x extent of our image to $[-2, 2]$ on the real axis in the complex plane.
2. Map the y extent of our image to $[-2j, 2j]$ on the imaginary axis in the complex plane.
3. Find the escape time of the complex number corresponding to each pixel in the image.
4. Compute a color for a given escape time.

Task: Visualize the Mandelbrot set as described above with n_t defined as a global variable at the beginning of your JS file. You will find that a simple linear interpolation between the colors for $n = 1$ and $n = n_t$ will lead to a bland visualization as seen in Figure 2 (a). You are encouraged to think about and implement an appropriate coloring scheme for the escape times. You may implement any coloring scheme that sufficiently distinguishes between different escape times.

Tips and Notes:
– As you may imagine, this computation can be expensive. Pick an n_t that is feasible for your machine and leads to an interesting visualization.
– Make liberal use of the `map_point()` function. Points 1, 2, and 4 above lend themselves simply to its use.

Files: `mandelbrot.html, mandelbrot.js`

Submission Instructions

You will turn in your code via Canvas. Please submit a single zip file containing all the required files. The zip file must be named using your HU ID. For example, if my ID is aa04049, I would name my zip file as "04049.zip". Please **do not** include your name or initials anywhere in your submission. Also, do not create a .rar or .tar file – please create a .zip file.
Make sure to follow the specified file naming convention. Also make sure that you create separate files for each problem as specified. If you do not follow the submission instructions it is possible that you will get a small point deduction.